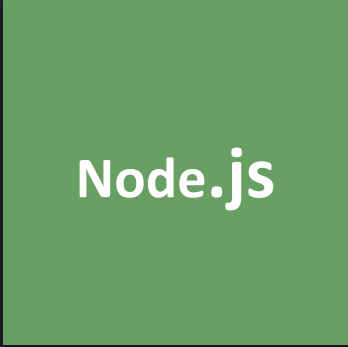


A solid yellow square containing the text 'JS' in a bold, dark blue, sans-serif font.

JS

A solid green square containing the text 'Node.js' in a white, sans-serif font.

Node.js

Fundamental JavaScript & Node.js

Agenda — Alur Belajar

SEKSI 1

Konsep Dasar

Node.js vs PHP — Cara Proses Request

Pengenalan JavaScript

Karakteristik JS + Analogi

SEKSI 2

Persiapan

Install Node.js (Ubuntu & Windows)

Tools — VS Code + Extensions

Node.js & NPM — Hello World

SEKSI 3

JavaScript Fundamentals

Variabel & Tipe Data + PRAKTIK 1

Operator & Control Flow + PRAKTIK 2

Fungsi + PRAKTIK 3

Array & Object + PRAKTIK 4

ES6+ & Async/Await + PRAKTIK 5

SEKSI 4

Node.js Backend

File System & Express.js

PRAKTIK 6 — File System JSON

PRAKTIK 7 — REST API Todo

Best Practices & Roadmap

SEKSI 1

Konsep Dasar

Memahami 'apa' dan 'mengapa' sebelum menulis satu baris kode

1.1 | Node.js vs PHP — Cara Memproses Request

Perbedaan arsitektur yang menentukan performa & skalabilitas

PHP + Apache/Nginx



Masalah: Model Blocking/Menunggu

100 request serentak = butuh 100 worker aktif (PHP-FPM pm.max_children)
Worker IDLE tapi tetap "ditempati" selama menunggu DB / file / API

Node.js (Event Loop)



Keunggulan: Non-blocking I/O

10.000 request serentak tetap 1 proses — CPU tidak pernah idle menunggu I/O
Ideal: REST API, real-time (chat), WebSocket, microservice

1.2 | Pengenalan JavaScript

Apa itu JavaScript?

JavaScript adalah bahasa pemrograman yang awalnya dirancang untuk membuat halaman web menjadi interaktif di browser. Diciptakan oleh Brendan Eich pada tahun 1995

- Server — Node.js memungkinkan JS berjalan di backend
- Desktop — Electron (VS Code, Slack, Figma dibangun dengan ini)
- IoT — perangkat seperti Raspberry Pi
- Mobile — React Native untuk aplikasi iOS/Android

Karakteristik Utama:

- Dynamically typed — tidak perlu deklarasi tipe
 - JavaScript hanya punya satu thread — artinya hanya bisa mengerjakan satu hal dalam satu waktu. Tapi ia tidak memblokir eksekusi berkat Event Loop.
- Single-threaded + Event Loop
 - JavaScript hanya punya satu thread — artinya hanya bisa mengerjakan satu hal dalam satu waktu. Tapi ia tidak memblokir eksekusi berkat Event Loop.
- Interpreted / JIT compiled
 - Code JS tidak dikompilasi dulu sebelum dijalankan seperti C/Java. Browser langsung membaca dan menjalankan kode baris per baris (interpreted).
- Multi-paradigm
 - JavaScript adalah bahasa yang fleksibel, cepat dikuasai untuk hal sederhana, tapi punya kedalaman konsep (async, closure, prototype chain) yang butuh waktu untuk benar-benar dipahami
 - OOP, Functional, Procedural

```
// Di terminal Node.js:
console.log('Hello, World!');
console.log('Versi Node:', process.version);

// Di browser (F12 → Console):
alert('Halo Browser!');

// Template string
const nama = 'Budi';
console.log(`Halo, ${nama}!`); // Halo, Budi!
```

1.3 | Karakteristik JavaScript — Detail & Analogi

Memahami cara kerja JS dari dalam sebelum menulis kode

1. Dynamically Typed

Tipe variabel tidak ditetapkan saat menulis kode.

Tipe ditentukan otomatis saat runtime berdasarkan nilai yang disimpan.

Kebalikannya: Static Typed (Java, C#, Go) — tipe HARUS dideklarasikan dulu.

Analogi: Tas Ransel Serbaguna

Static (Java): 3 tas berlabel khusus:
"Tas Buku", "Tas Makanan", "Tas Pakaian"
Isi yang salah → **ERROR** sebelum jalan!

Dynamic (JS): 1 ransel bebas — isi buku sekarang, botol nanti, laptop besok. OK!
Ransel tidak peduli — semua muat!

```
// JavaScript
let x = 42;           // Number
x = 'hello';          // String — OK!
x = [1,2,3];          // Array — OK!
typeof x;             // 'object'
```

```
// Java:
int x = 42;
x = "hi"; // ERROR
```

2. Single-threaded + Event Loop

JS hanya punya 1 thread (jalur kerja).

Event Loop membuatnya menangani banyak hal seolah bersamaan — tidak pernah berdiam menunggu I/O selesai.

Analogi: Barista Kopi Sendirian

Barista = 1 orang (single-thread)

Pelanggan A pesan kopi → barista mulai

Sambil menunggu: layani B, C, D...
Timer bunyi → barista kembali ke A.

Event Loop = memanggil!

```
console.log('1. Pesanan diterima');
setTimeout(() => {
  console.log('3. Kopi siap!'); // 2s
}, 2000);
console.log('2. Layani yang lain');
// Output: 1 → 2 → 3 (bukan 1→3→2!)
```

3. Interpreted / JIT Compiled

Interpreted: kode berjalan baris per baris tanpa kompilasi dulu.

JIT (Just-In-Time): V8 memantau fungsi yang sering dipanggil, lalu mengompilasinya ke machine code → makin cepat.

Analogi: Penerjemah Bahasa

Compiled (C/Go): kode diterjemahkan penuh menjadi machine code sebelum dijalankan. Proses build lebih lama, tapi eksekusi sangat cepat.

Interpreted: kode dibaca dan dijalankan baris demi baris secara langsung saat runtime

SEKSI 2

Persiapan

Install Node.js + tools

2.1 | Install Node.js

Ubuntu / WSL2 & Windows — langkah per langkah

Ubuntu / Debian / WSL2

```
# === Metode 1: NVM (REKOMENDASI) ===
# Keuntungan: ganti versi mudah, tanpa sudo
curl -o-
https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh
| bash

# Tutup & buka terminal baru, lalu:
nvm install --lts      # pasang versi LTS terbaru
nvm use --lts
nvm alias default node

# Verifikasi instalasi:
node --version        # v20.x.x (LTS)
npm --version         # 10.x.x

# Ganti versi kapan saja:
nvm install 18        # install v18
nvm use 18            # pakai v18
nvm list              # lihat semua versi

# === Metode 2: NodeSource (via apt) ===
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash
-
sudo apt install nodejs -y

# Metode 3: apt bawaan (versi lama — tidak disarankan)
sudo apt install nodejs npm -y    # mungkin v12!
```

Windows 10 / 11

```
# === Metode 1: Installer Resmi ===
# 1. Buka: nodejs.org → klik tombol LTS
# 2. Download file .msi → jalankan installer
# 3. Ikuti wizard: Next → Next → Finish
# 4. Centang: 'Automatically install tools'

# Verifikasi di Command Prompt / PowerShell:
node --version
npm --version

# === Metode 2: NVM for Windows (REKOMENDASI) ===
# Download: github.com/coreybutler/nvm-windows
# Jalankan nvm-setup.exe, lalu:
nvm install lts
nvm use lts
nvm list

# === Metode 3: Winget ===
winget install OpenJS.NodeJS.LTS

# Troubleshoot: jika 'node' tidak dikenali
# → Restart terminal atau PC
# → PATH sudah ditambahkan otomatis oleh installer
```

2.2 | Tools — VS Code & Extensions Wajib

SETUP

Editor + plugin yang membuat coding JavaScript jauh lebih nyaman

VS Code — Download

code.visualstudio.com

Windows: download .exe installer

Ubuntu: sudo snap install code --classic
atau download .deb dari website

Extensions Wajib Install (Ctrl+Shift+X)

ESLint

dbaeumer.vscode-eslint

Deteksi error & bug JS secara real-time. Wajib untuk semua project JS.

Node.js Ext Pack

waderyan.nodejs-extension-pack

Bundel tools Node.js: snippets, debugger, npm intellisense, dll.

settings.json — Pengaturan Rekomendasi

```
// Ctrl+Shift+P → 'Open User Settings (JSON) '  
{  
  "editor.formatOnSave": true,  
  "editor.defaultFormatter": "esbenp.prettier-vscode",  
  "editor.tabSize": 2,  
  "editor.fontSize": 14,  
  "terminal.integrated.defaultProfile.windows": "PowerShell",  
  "files.autoSave": "afterDelay",  
  "javascript.updateImportsOnFileMove.enabled": "always"  
}
```

Prettier

esbenp.prettier-vscode

Auto-format kode saat save. Konsistensi indentasi & gaya kode.

GitLens

eamodio.gitlens

Lihat siapa yang ubah baris kode, history commit inline di editor.

REST Client

humao.rest-client

Test API langsung dari VS Code — buat file .http, klik 'Send Request'.

Thunder Client

rangav.vscode-thunder-client

Postman versi ringan di dalam VS Code — test REST API dengan GUI.

2.3 | Node.js & NPM — Hello World

PRAKTIK

Node.js — Konsep Kunci

Runtime JS berbasis V8 Chrome
1 proses, non-blocking event loop
Built-in: fs, http, path, os, crypto

Global objects di Node.js:

process — info proses & env vars
__dirname — direktori file saat ini
require() — import modul
module.exports — export modul

NPM — Node Package Manager

```
npm init -y           # buat package.json
npm install express    # install paket
npm install nodemon -D # dev dependency
npm run dev            # jalankan script
```

Langkah Pertama — Buat Project

```
# 1. Buat folder & masuk
mkdir belajar-node && cd belajar-node

# 2. Inisiasi project
npm init -y

# 3. Buat file hello.js
console.log('Hello dari Node.js!');
console.log('Versi :', process.version);
console.log('OS    :', process.platform);
console.log('Dir   :', __dirname);

# 4. Jalankan!
node hello.js

# Output:
# Hello dari Node.js!
# Versi : v20.x.x
# OS    : linux / win32 / darwin

# 5. Buat script di package.json:
# "scripts": { "start": "node hello.js" }
npm start
```

SEKSI 3

JavaScript Fundamentals

Teori + praktik — variabel → async/await

3.1 | Variabel & Tipe Data

Deklarasi Variabel

```
// var - function-scoped, HINDARI
var nama = 'Budi';

// let - block-scoped, bisa diubah
let umur = 25;
umur = 26;          // OK

// const - block-scoped, tidak bisa reassign
const PI = 3.14159;
// PI = 3;          // TypeError - ERROR!

// Tipe berubah otomatis (dynamically typed)
let x = 42;          // Number
x = 'hello';         // sekarang String - valid!
typeof x;            // 'string'
```

7 Tipe Data Primitif

```
// String - teks
let s = 'Hello';    // atau "Hello" atau `Hello`

// Number - int & float dalam 1 tipe
let n = 42;  let f = 3.14;  let nan = NaN;

// Boolean
let b = true;  let c = false;

// null - kosong disengaja
let kosong = null;

// undefined - belum diberi nilai
let belum;    // undefined

// BigInt - angka sangat besar
let besar = 9007199254740991n;

// Symbol - identifier unik
let id = Symbol('id');
```

PRAKTIK 1 | Variabel & Tipe Data

PRAKTIK 1

Eksplorasi typeof, konversi tipe, dan scope

Buat file: latihan-variabel.js

```
// === SOAL 1: typeof & konversi ===
let angka = '42';
let desimal = '3.14';
let boolStr = 'true';

console.log(typeof angka);           // ?
console.log(Number(angka));          // ?
console.log(Number(desimal));        // ?
console.log(Boolean(boolStr));       // ? (hati-hati!)
console.log(Boolean(''));            // ?
console.log(Boolean(0));              // ?

// === SOAL 2: const vs let scope ===
const MAX = 100;
let total = 0;
{
  let lokal = 50;    // hanya hidup di block ini
  total = lokal + 25;
}
console.log(total);  // ?
// console.log(lokal); // ReferenceError - coba uncomment!

// === SOAL 3: string methods ===
const kalimat = ' Belajar JavaScript Itu Seru! ';
console.log(kalimat.trim().toLowerCase());
console.log(kalimat.trim().split(' ').length);
console.log(kalimat.includes('JavaScript'));
```

Output yang Diharapkan

```
string
42
3.14
true    // 'true' string = truthy!
false
false
---
75
---
  belajar javascript itu seru!
4
true
```

Cara Menjalankan

```
node latihan-variabel.js

# Atau coba REPL interaktif:
node
> typeof 'hello'
> Number('42')
> Boolean('')
> .exit
```

3.2 | Operator & Control Flow

Membuat keputusan dan mengulang logika

Operator Penting

```
// Perbandingan - WAJIB pakai ===
5 == '5'    // true  ← HINDARI, type coercion!
5 === '5'   // false ← SELALU gunakan ini
5 !== 3     // true

// Logika
true && false // false (AND)
true || false // true  (OR)
!true        // false (NOT)

// Modern (ES2020)
null ?? 'default' // 'default' (nullish)
user?.address?.city // aman walau undefined
umur>=17 ? 'Dewasa':'Anak' // ternary
```

if / else & Loop

```
const nilai = 85;
if (nilai >= 90) { console.log('A'); }
else if (nilai >= 75) { console.log('B'); }
else if (nilai >= 60) { console.log('C'); }
else { console.log('D'); }

// for loop klasik
for (let i = 1; i <= 5; i++) { console.log(i); }

// for...of - iterasi array
for (const item of ['a','b','c']) { console.log(item); }

// while loop
let i = 0;
while (i < 3) { console.log(i++); }

// for...in - iterasi key object
const user = { nama:'Ani', umur:22 };
for (const key in user) { console.log(key, user[key]); }
```

PRAKTIK 2 | Control Flow — Kalkulator Nilai & FizzBuzz

PRAKTIK 2

if/else, loop, dan logika percabangan

Buat file: kontrol.js

```
// === SOAL 1: Kalkulator Nilai ===
function hitungGrade(nilai) {
  if (nilai < 0 || nilai > 100) return 'Input tidak valid!';
  if (nilai >= 90) return 'A - Sangat Memuaskan';
  if (nilai >= 80) return 'B - Memuaskan';
  if (nilai >= 70) return 'C - Cukup';
  if (nilai >= 60) return 'D - Kurang';
  return 'E - Tidak Lulus';
}
console.log(hitungGrade(95)); // A
console.log(hitungGrade(72)); // C
console.log(hitungGrade(150)); // tidak valid

// === SOAL 2: FizzBuzz 1-30 ===
// Kode JavaScript melakukan perulangan angka dari 1 sampai
30
// mulai dari 1
// simpan ke variable i
// selama i <= 30
// ulang terus
// setiap selesai loop → i++ (tambah 1)

for (let i = 1; i <= 30; i++) {
  if (i % 15 === 0) console.log(`${i}: FizzBuzz`);
  else if (i % 3 === 0) console.log(`${i}: Fizz`);
  else if (i % 5 === 0) console.log(`${i}: Buzz`);
  else console.log(i);
}
```

Output

```
A - Sangat Memuaskan
C - Cukup
Input tidak valid!

1
2
3: Fizz
4
5: Buzz
6: Fizz
7
8
9: Fizz
10: Buzz ...
15: FizzBuzz...
30: FizzBuzz
```

Cara Menjalankan

```
node kontrol.js

# Coba juga di Node REPL:
node
> for(let i=1;i<=10;i++)
...
console.log(i%3===0?'Fizz'
:i)
```

3.3 | Fungsi (Functions)

Blok code yang dapat digunakan kembali

Deklarasi & Arrow Function

```
// Declaration - hoisted (bisa dipanggil sebelum definisi)
function sapa(nama, sapaan = 'Halo') {
  return `${sapaan}, ${nama}!`;
}
console.log(sapa('Budi'));           // Halo, Budi!
console.log(sapa('Ani', 'Selamat')); // Selamat, Ani!

// Function Expression - tidak hoisted
const tambah = function(a, b) { return a + b; };

// Arrow Function (ES6) - sintaks ringkas
const kali    = (a, b) => a * b;
const kuadrat = x => x ** 2;
const greet   = () => 'Hello World!';

// Rest parameters
const sum = (...nums) => nums.reduce((a,b) => a+b, 0);
sum(1, 2, 3, 4, 5); // 15
```

Higher Order Function & Closure

```
// Higher Order Function - fungsi sebagai argumen
function operasi(a, b, fn) { return fn(a, b); }
operasi(10, 3, (a,b) => a + b); // 13
operasi(10, 3, (a,b) => a * b); // 30
operasi(10, 3, Math.max);      // 10

// Closure - fungsi mengingat scope asalnya
function buatCounter(mulaiDari = 0) {
  let count = mulaiDari;
  return {
    tambah: () => ++count,
    kurangi: () => --count,
    reset: () => { count = mulaiDari; },
    nilai: () => count,
  };
}
const c = buatCounter(10);
c.tambah(); c.tambah(); // count = 12
c.kurangi(); // count = 11
console.log(c.nilai()); // 11
```

Arrow function, default param, higher order function, closure

Buat file: fungsi.js

```
// === SOAL 1: Kalkulator lengkap ===
const kalkulator = {
  tambah: (a, b) => a + b,
  kurangi: (a, b) => a - b,
  kali: (a, b) => a * b,
  bagi: (a, b) => b !== 0 ? a / b : 'Error: dibagi nol!',
  pangkat: (a, b) => a ** b,
};

console.log(kalkulator.tambah(10, 5)); // 15
console.log(kalkulator.bagi(10, 0)); // Error: dibagi nol!
console.log(kalkulator.pangkat(2, 8)); // 256

// === SOAL 2: HOF — terapkan ke array ===
// Higher-order function (HOF), function yang menerima function lain sebagai parameter
function terapkan(arr, fn) { return arr.map(fn); }
console.log(terapkan([1,2,3,4], x => x**2)); // [1,4,9,16]
console.log(terapkan([1,2,3,4], x => x%2===0)); // [F,T,F,T]

// === SOAL 3: Closure — rekening bank ===
// Closure, Function masih bisa mengingat variable lama meskipun function pembuatnya sudah selesai
function buatRekening(saldoAwal) {
  let saldo = saldoAwal;
  return {
    setor: (n) => { saldo += n; console.log(`Saldo: ${saldo}`); },
    tarik: (n) => { if (n > saldo) { console.log('Saldo kurang!'); return; }
                  saldo -= n; console.log(`Saldo: ${saldo}`); },
    cek: () => saldo,
  };
}

const rek = buatRekening(500000);
rek.setor(200000); // Saldo: 700000
rek.tarik(1000000); // Saldo kurang!
rek.tarik(100000); // Saldo: 600000
```

Cara Menjalankan & Output

```
node fungsi.js

# Output Soal 1:
# 15
# Error: dibagi nol!
# 256

# Output Soal 2:
# [ 1, 4, 9, 16 ]
# [ false, true, false, true ]

# Output Soal 3:
# Saldo: 700000
# Saldo kurang!
# Saldo: 600000
```

3.4 | Array & Object

Struktur data utama JavaScript

Array Methods Fungsional

```
const mahasiswa = [
  { nama:'Budi', nilai:85 }, { nama:'Ani', nilai:92 },
  { nama:'Caca', nilai:67 }, { nama:'Dani', nilai:78 },
];

// map - transformasi setiap elemen
mahasiswa.map(m => m.nama); // ['Budi','Ani',...]

// filter - pilih yang memenuhi syarat
mahasiswa.filter(m => m.nilai >= 80); // Budi, Ani

// reduce - akumulasi
const rata = mahasiswa.reduce((acc,m)=>acc+m.nilai,0)
  / mahasiswa.length; // 80.5

mahasiswa.find(m => m.nama === 'Ani');
mahasiswa.sort((a,b) => b.nilai - a.nilai); // desc
mahasiswa.includes({ nama:'Budi' }); // false (ref!)
```

Object — Manipulasi & Spread

```
const user = {
  nama: 'Ani', umur: 22,
  alamat: { kota: 'Jakarta', kodePos: '10110' },
  hobi: ['coding','membaca'],
  sapa() { return `Hi, ${this.nama}!`; }
};

// Akses properti
user.nama; // 'Ani'
user['umur']; // 22
user.alamat.kota; // 'Jakarta'
user.sapa(); // 'Hi, Ani!'

// Object methods
Object.keys(user); // ['nama','umur',...]
Object.values(user); // ['Ani', 22, ...]
Object.entries(user); // [['nama','Ani'],...]

// Spread - salin & ubah tanpa mutasi
const updated = { ...user, umur:23, role:'admin' };
const merged = { ...obj1, ...obj2 };
```

map, filter, reduce, sort, grouping

Buat file: mahasiswa.js

```
const data = [
  { nama: 'Budi', jurusan: 'TI', nilai: [80, 90, 85] },
  { nama: 'Ani', jurusan: 'SI', nilai: [95, 88, 92] },
  { nama: 'Caca', jurusan: 'TI', nilai: [60, 72, 65] },
  { nama: 'Dani', jurusan: 'SI', nilai: [78, 82, 80] },
  { nama: 'Eka', jurusan: 'TI', nilai: [91, 88, 95] },
];

// SOAL 1: Hitung rata-rata tiap mahasiswa
const withAvg = data.map(m => ({
  ...m,
  rata: m.nilai.reduce((a, b) => a + b, 0) / m.nilai.length
}));
withAvg.forEach(m => console.log(`${m.nama}: ${m.rata.toFixed(1)}`));

// SOAL 2: Filter lulus (rata >= 75)
const lulus = withAvg.filter(m => m.rata >= 75);
console.log('Lulus:', lulus.map(m => m.nama));

// SOAL 3: Ranking dari nilai tertinggi
const ranking = [...withAvg].sort((a, b) => b.rata - a.rata);
ranking.forEach((m, i) => console.log(`${i+1}. ${m.nama} - ${m.rata.toFixed(1)}`));

// SOAL 4: Grouping per jurusan
const perJurusan = data.reduce((acc, m) => {
  if (!acc[m.jurusan]) acc[m.jurusan] = [];
  acc[m.jurusan].push(m.nama);
  return acc;
}, {});
console.log(perJurusan);
```

Output yang Diharapkan

```
Budi: 85.0    Ani: 91.7
Caca: 65.7    Dani: 80.0    Eka: 91.3
```

```
Lulus: ['Budi', 'Ani', 'Dani', 'Eka']
```

```
1. Ani - 91.7
2. Eka - 91.3
3. Budi - 85.0
4. Dani - 80.0
5. Caca - 65.7
```

```
{ TI: ['Budi', 'Caca', 'Eka'],
  SI: ['Ani', 'Dani'] }
```

3.5 | ES6+ Modern Syntax & Async/Await

Fitur modern yang wajib dikuasai

Destructuring & Spread

```
// Object destructuring + rename + default
const { nama, umur = 18, alamat: { kota } } = user;

// Array destructuring + skip + rest
const [a, , c, ...rest] = [1,2,3,4,5];
// a=1  c=3  rest=[4,5]

// Template literals
const pesan = `Halo ${nama}, kota: ${kota}`;

// Spread
const arr2 = [...arr1, 4, 5];
const obj2 = { ...user, role: 'admin' };

// Modules (CommonJS Node.js)
// math.js:  module.exports = { add };
// app.js:    const { add } = require('./math');
```

Rule: Semua fungsi yang menggunakan await harus dideklarasikan async. await hanya valid di dalam async function.

Async / Await

```
// Promise - operasi asynchronous
const janji = new Promise((resolve, reject) => {
  setTimeout(() => resolve('Selesai!'), 1000);
});
janji.then(v => console.log(v)); // 1 detik kemudian

// async/await - cara modern, lebih bersih
async function ambilData(url) {
  try {
    const res  = await fetch(url);
    const data = await res.json();
    return data;
  } catch (err) {
    console.error('Gagal:', err.message);
  }
}

// Parallel - jalankan serentak
const [users, posts] = await Promise.all([
  ambilData('/api/users'),
  ambilData('/api/posts'),
]);

// Delay / sleep helper
const delay = ms => new Promise(r => setTimeout(r,ms));
await delay(2000); // tunggu 2 detik
```

Buat file: fetch-api.js

```
// Node.js 18+ sudah punya fetch built-in!
// Node < 18: npm install node-fetch

// === SOAL 1: Ambil 1 user dari API publik ===
async function ambilUser(id) {
  const res = await fetch(`https://jsonplaceholder.typicode.com/users/${id}`);
  if (!res.ok) throw new Error(`HTTP error: ${res.status}`);
  const user = await res.json();
  console.log(`Nama : ${user.name}`);
  console.log(`Email : ${user.email}`);
  console.log(`Kota : ${user.address.city}`);
}

// === SOAL 2: Ambil posts lalu filter ===
async function postsByUser(userId) {
  const res = await fetch('https://jsonplaceholder.typicode.com/posts');
  const posts = await res.json();
  const milik = posts.filter(p => p.userId === userId);
  console.log(`User ${userId} punya ${milik.length} post`);
  milik.slice(0, 3).forEach(p => console.log(' -', p.title));
}

// === SOAL 3: Parallel fetch ===
async function main() {
  console.log('=== User ==='); await ambilUser(1);
  console.log('\n=== Posts ==='); await postsByUser(1);
  console.log('\n=== Parallel ===');
  await Promise.all([ambilUser(2), ambilUser(3)]);
}
main().catch(console.error);
```

Output yang Diharapkan

```
=== User ===
Nama   : Leanne Graham
Email  : Sincere@april.biz
Kota   : Gwenborough
```

```
=== Posts ===
User 1 punya 10 post
- sunt aut facere ...
- qui est esse
```

```
=== Parallel ===
Nama   : Ervin Howell
Nama   : Clementine Bauch
```

Cara Menjalankan

```
# Node.js 18+:
node fetch-api.js

# Node.js < 18:
npm install node-fetch
# tambah di baris 1:
# import fetch from 'node-fetch';
# (package.json: type:module)

# Cek versi:
node --version
```

SEKSI 4

Node.js Backend

File System, Express.js, dan membangun REST API

4.1 | File System & Express.js

I/O file dan framework web Node.js

fs/promises — Baca & Tulis File

```
const fs = require('fs/promises');
const path = require('path');

// Tulis file JSON
const data = { users: [{ id:1, nama:'Budi' }] };
await fs.writeFile('data.json',
JSON.stringify(data,null,2));

// Baca file JSON
const raw = await fs.readFile('data.json', 'utf8');
const obj = JSON.parse(raw);
console.log(obj.users);

// Append ke file log
const log = `[${new Date().toISOString()}] Event\n`;
await fs.appendFile('app.log', log);

// Cek & buat direktori
await fs.mkdir('uploads', { recursive: true });
```

Express.js — REST API Cepat

```
# npm install express nodemon
const express = require('express');
const app = express();
app.use(express.json());

let items = [{ id:1, nama:'Laptop' }];

app.get('/items', (req, res) => res.json(items));

app.get('/items/:id', (req, res) => {
  const item = items.find(i => i.id == req.params.id);
  item ? res.json(item)
    : res.status(404).json({ error:'Not found' });
});

app.post('/items', (req, res) => {
  const item = { id: Date.now(), ...req.body };
  items.push(item);
  res.status(201).json(item);
});

app.put('/items/:id', (req, res) => {
  const item = items.find(i => i.id == req.params.id);
  if (!item) return res.status(404).json({error:'Not found'});
  Object.assign(item, req.body); res.json(item);
});

app.delete('/items/:id', (req, res) => {
  items = items.filter(i => i.id != req.params.id);
  res.status(204).send();
});

app.listen(3000, () => console.log('port 3000'));
```

Baca, tulis, update, hapus data ke file .json

Buat file: db.js + app.js

```
// db.js — helper baca/tulis JSON
const fs = require('fs/promises');
const FILE = 'users.json';
async function baca() { const r=await fs.readFile(FILE,'utf8'); return JSON.parse(r); }
async function simpan(data) { await fs.writeFile(FILE, JSON.stringify(data,null,2)); }
module.exports = { baca, simpan };
```

```
// — app.js —
const { baca, simpan } = require('./db');
```

```
// CREATE
async function tambahUser(nama, email) {
  const db = await baca();
  const user = { id: Date.now(), nama, email };
  db.users.push(user);
  await simpan(db);
  console.log('Ditambah:', user);
}
```

```
// READ
async function lihatSemua() {
  const db = await baca();
  console.table(db.users);
}
```

```
// UPDATE
async function updateUser(id, data) {
  const db = await baca();
  const user = db.users.find(u => u.id === id);
  if (!user) return console.log('Tidak ditemukan');
  Object.assign(user, data); await simpan(db);
  console.log('Diupdate:', user);
}
```

```
// DELETE
async function hapusUser(id) {
  const db = await baca();
  db.users = db.users.filter(u => u.id !== id);
  await simpan(db); console.log('Dihapus:', id);
}
```

```
async function main() {
  await simpan({ users: [] });
  await tambahUser('Budi','budi@mail.com');
  await tambahUser('Ani', 'ani@mail.com');
  await lihatSemua();
}
main().catch(console.error);
```

Cara Menjalankan

```
node app.js
```

```
# Output console.table:
```

```
#
# (idx)  nama      email
#
# 0      'Budi'    'budi@mail.com'
# 1      'Ani'     'ani@mail.com'
```

```
# File users.json dibuat otomatis:
cat users.json
```

Tantangan:

1. Cari user by nama
2. Tambah field createdAt otomatis
3. Cegah email duplikat

PRAKTIK 7 | Mini Project — REST API Todo dengan Express

Project akhir: CRUD API lengkap + test dengan curl / Thunder Client

Struktur & Setup

```
todo-api/
├- index.js
├- routes/
│  └- todos.js
└- package.json

# Setup:
mkdir todo-api && cd todo-api
npm init -y
npm install express
npm install nodemon -D
```

package.json

```
{
  "name": "todo-api",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "start": "node index.js"
  },
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

GET	/todos — Semua todo
GET	/todos/:id — By ID
POST	/todos — Buat todo
PUT	/todos/:id — Update
DELETE	/todos/:id — Hapus

routes/todos.js

```
const router = require('express').Router();
let todos = []; let nextId = 1;

router.get('/', (req, res) => res.json(todos));

router.get('/:id', (req, res) => {
  const todo = todos.find(t => t.id == req.params.id);
  todo ? res.json(todo) : res.status(404).json({error:'Not found'});
});

router.post('/', (req, res) => {
  const { title } = req.body;
  if (!title) return res.status(400).json({error:'title required'});
  const todo = { id:nextId++, title, done:false,
    createdAt: new Date().toISOString() };
  todos.push(todo); res.status(201).json(todo);
});

router.put('/:id', (req, res) => {
  const todo = todos.find(t => t.id == req.params.id);
  if (!todo) return res.status(404).json({error:'Not found'});
  Object.assign(todo, req.body); res.json(todo);
});

router.delete('/:id', (req, res) => {
  todos = todos.filter(t => t.id != req.params.id);
  res.status(204).send();
});

module.exports = router;
```

index.js

```
const express = require('express');
const todos = require('./routes/todos');
const app = express();
app.use(express.json());
app.use('/todos', todos);
app.listen(3000, () => console.log('Todo API: port 3000'));
```

Cara Menjalankan & Test

```
npx nodemon index.js

# GET semua:
curl http://localhost:3000/todos

# POST buat todo:
curl -X POST localhost:3000/todos \
  -H 'Content-Type: application/json' \
  -d '{"title":"Belajar Node.js"}'

# PUT update:
curl -X PUT localhost:3000/todos/1 \
  -H 'Content-Type: application/json' \
  -d '{"done":true}'

# DELETE:
curl -X DELETE localhost:3000/todos/1

# Atau gunakan Thunder Client di VS Code!
```

Terima Kasih!

Selamat belajar dan terus berlatih!